



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Институт (Филиал) № 8 «Компьютерные науки и прикладная математика» Кафедра 806

Группа М8О-410Б-20 Направление подготовки 02.03.02 Фундаментальная
информатика и информационные технологии

Профиль Информатика и компьютерные науки

Квалификация бакалавр

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

На тему: Применение алгоритмов машинного обучения для создания
модели рекомендательной системы фильмов.

Автор ВКРБ Мигурский Иван Феликсович (_____) (фамилия, имя, отчество полностью)

Руководитель Филимонов Александр Борисович (_____) (фамилия, имя, отчество полностью)

К защите допустить

Заведующий кафедрой 806 Крылов Сергей Сергеевич (_____) (№ каф) (фамилия, имя, отчество полностью)

_____ 2024г.

Москва 2024

РЕФЕРАТ

Выпускная квалификационная работа бакалавра содержит 46 страниц текста, 21 рисунок и 1 приложения. Список литературы состоит из 29 пунктов.

МАШИННОЕ ОБУЧЕНИЕ, РЕКОМЕНДАТЕЛЬНАЯ СИСТЕМА, ВЕРОЯТНОСТНАЯ МАТРИЧНАЯ ФАКТОРИЗАЦИЯ, БАЙЕСОВСКАЯ ВЕРОЯТНОСТНАЯ МАТРИЧНАЯ ФАКТОРИЗАЦИЯ, КРУПНОМАСШТАБНАЯ ПАРАЛЛЕЛЬНАЯ КОЛЛАБОРАТИВНАЯ ФИЛЬТРАЦИЯ, СРЕДНЕКВАДРАТИЧНАЯ ОШИБКА

Объектом разработки являются алгоритмы коллаборативной фильтрации.

Цель работы: алгоритмизация построения систем коллаборативной фильтрации на языке программирования Python с применением эффективных методов и технологий машинного обучения.

Достижение поставленной цели предполагает решение следующих задач:

- 1) анализ современных подходов к решению задач коллаборативной фильтрации;
- 2) построение трех моделей коллаборативной фильтрации: на основе вероятностной матричной факторизации, байесовской вероятностной матричной факторизации и крупномасштабной параллельной коллаборативной фильтрации;
- 3) сравнительный анализ эффективности разработанных алгоритмов.

Основные результаты выполненной ВКР:

- 1) программно реализован алгоритм построения модели вероятностной матричной факторизации;
- 2) программно реализован алгоритм построения модели байесовской вероятностной матричной факторизации;
- 3) программно реализован алгоритм построения модели крупномасштабной параллельной коллаборативной фильтрации;

4) разработан алгоритм и соответствующая программа построения трёх моделей рекомендательных систем на датасете и их сравнительного анализа.

Применение полученных результатов в конкретных практических разработках позволяет повысить эффективность проектируемых рекомендательных систем.

СОДЕРЖАНИЕ

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	6
ВВЕДЕНИЕ	7
1. Рекомендательные системы	9
1.1. Развитие рекомендательных систем	9
1.2. Обзор существующих решений	9
1.3. Классификация рекомендательных систем	10
2. Вероятностная матричная факторизация	15
2.1. Общие характеристики вероятностной матричной факторизации	15
2.2. Вероятностная матричная факторизация (PMF)	17
2.3. Реализация модели PMF на python	19
2.4. Экспериментальные результаты	20
3. Байесовская вероятностная матричная факторизация	22
3.1. Общие характеристики метода Байесовской вероятностной матричной факторизации	22
3.2. Модель байесовской вероятностной матричной факторизации	24
3.3. Прогнозы	25
3.4. Алгоритм MCMC	26
3.5. Реализация модели BPMF на python	28
3.6. Экспериментальные результаты	30
4. Крупномасштабная параллельная коллаборативная фильтрация	32
4.1. Общие характеристики метода крупномасштабной параллельной коллаборативной фильтрации	32
4.2. ALS со взвешенной λ -регуляризацией	34
4.3. Реализация модели ALS на python	36
4.4. Экспериментальные результаты	37
5. Сравнение моделей рекомендательных систем	38
5.1. Датасет MovieLens	38

5.2. Сравнение результатов работы моделей.....	38
Заключение	42
Список источников	43
ПРИЛОЖЕНИЕ А Ссылка на код	46

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

PMF – Probabilistic Matrix Factorization

BPMF – Bayesian Probabilistic Matrix Factorization

ALS – Alternating Least Squares

SVD – Singular Value Decomposition

MCMC – Markov Chain Monte Carlo

RMSE – Root Mean Squared Error

ВВЕДЕНИЕ

Последние десятилетия характеризуются бурным развитием сети Интернет: ежедневно в глобальной паутине генерируются и накапливаются колоссальные объемы информации. Пользователю приходится работать с этими данными: обрабатывать, систематизировать, находить релевантные для его информационных потребностей данные. Человеку сложно отобрать интересующую его информацию путем обычного просмотра, так как часто релевантная информация теряется среди больших объемов данных. В связи с этим, создаются инструменты, которые могут помочь человеку в поиске, предлагая тот контент, который предпочтителен для пользователя. Такие программные средства получили название рекомендательные системы.

Рекомендательные системы (англ. *recommender systems*) - программы и сервисы, которые анализируют интересы пользователей и пытаются предсказать, что именно будет наиболее интересно для конкретного пользователя в данный момент времени. Такие системы показывают предпочтительность контента для конкретного юзера на основе данных, указанных пользователем явно или на основе его взаимодействия с системой.

Технология создания рекомендательных систем в настоящее время активно развивается, поэтому исследования в этой области компьютерных технологий актуальны и представляют как теоретический, так и практический интерес.

Цель работы: алгоритмизация построения систем коллаборативной фильтрации на языке программирования Python с применением эффективных методов и технологий машинного обучения.

Рекомендательные системы нашли свое применение во многих сферах жизнедеятельности человека: поиске фильмов и научных статей, розничной торговле, социальных сетях, электронной коммерция, онлайн-банкинге и т.д. Подобная задача применима и к сфере рекомендации фильмов.

Достижение поставленной цели предполагает решение следующих задач:

- 1) анализ современных подходов к решению задач коллаборативной фильтрации;
- 2) построение трех моделей коллаборативной фильтрации: на основе вероятностной матричной факторизации, байесовской вероятностной матричной факторизации и крупномасштабной параллельной коллаборативной фильтрацией;
- 3) сравнительный анализ эффективности разработанных алгоритмов.

Выпускная квалификационная работа включает пять глав, заключение, список используемой литературы и приложение

В первой главе приведены общие положения теории рекомендательных систем. Вторая глава посвящена задаче коллаборативной фильтрации на основе вероятностной матричной факторизации, третья - коллаборативной фильтрации на основе байесовской вероятностной матричной факторизации, четвёртая - крупномасштабной параллельной коллаборативной фильтрации. В пятой главе дается сравнительный анализ трех выделенных моделей коллаборативной фильтрации.

1 РЕКОМЕНДАТЕЛЬНЫЕ СИСТЕМЫ

Рекомендательные системы – это программные средства, которые предсказывают, какие объекты (фильмы, музыка, книги, новости, веб-сайты и т.д.) будут интересны пользователю, если имеется определенная информация о его предпочтениях. Рекомендации формируются отдельно для каждого человека на основе прошлой активности и поведения остальных пользователей системы. Существуют два основных подхода к построению рекомендаций: на основе коллаборативной фильтрации и на основе фильтрации содержимого.

1.1 Развитие рекомендательных систем

Рекомендательные системы возникли в области машинного обучения в 50-х годах. В начале 1990-х коллаборативная фильтрация стала применяться как решение для борьбы с избыточной информацией в вебе. В 2006 году *Netflix* запустила соревнование *Netflix Prize*, целью которого было создание алгоритма рекомендаций, который смог бы улучшить результат действующего внутреннего алгоритма *CineMatch* на 10%. Это вызвало шквал активности в академических кругах и среди любителей. В то же время компания *Google* представила свою систему персонализации новостей *Google News* на основе истории кликов пользователей.

1.2 Обзор существующих решений

Рекомендательные системы уже интегрированы во множество веб-приложений, которые широко используются каждый день миллионами пользователей. Рассмотрим примеры крупнейших ресурсов, использующих рекомендательные механизмы:

LinkedIn - бизнес-ориентированная социальная сеть, предлагающая рекомендации людей, вакансий и групп.

Amazon - одна из крупнейших площадок интернет-торговли, использующая рекомендации на основе контента.

Last.fm - создает музыкальную станцию рекомендованных песен, наблюдая, какие группы и отдельные треки пользователь прослушивает на регулярной основе.

Pandora - использует метаданные песен и исполнителей порядка 400 атрибутов, чтобы сгенерировать станцию, которая воспроизводит музыку с похожими свойствами.

1.3 Классификация рекомендательных систем

В базовых подходах для рекомендательных систем могут использоваться два вида данных:

- 1) Информация о взаимодействии пользователей с объектами интереса
- 2) Информация, предоставленная самими пользователями, например, атрибуты, указанные в профиле или релевантные ключевые слова.

Первую группу методов чаще всего называют методами коллаборативной фильтрации, для методов второй группы обычно используется название рекомендаций на основе контента. Еще один тип систем рекомендаций - системы рекомендаций, основанные на знаниях. Некоторые рекомендательные системы могут объединять перечисленные выше аспекты; такие системы называются гибридными.

Коллаборативная фильтрация (англ. Collaborative filtering) вырабатывает рекомендации, основанные на модели предшествующего поведения пользователя. Эта модель может быть построена исключительно на основе поведения данного пользователя или – что более эффективно – с учетом поведения других пользователей со сходными характеристиками. Коллаборативная фильтрация использует два разных типа входных данных, как показано на рисунке 1.1: множество пользователей и множество объектов интереса.



Рисунок 1.1 - Схема реализации коллаборативной фильтрации

Отношения между пользователями и объектами интереса обычно выражаются при помощи оценок, предоставляемых пользователями, и использующихся в последующих сессиях для прогнозирования оценок, которые пользователь (в нашем случае пользователь P_a) мог бы поставить неоцененным объектам интереса. Если предположить, что пользователь P_a в настоящее время взаимодействует с коллаборативной системой рекомендаций, первым делом система должна идентифицировать ближайших соседей (пользователей с аналогичным поведением, как и P_a) а затем экстраполировать из рейтингов похожих пользователей рейтинг пользователя P_a . Два основных подхода к коллаборативной фильтрации - это *user-based* коллаборативная фильтрация и *item-based* коллаборативная фильтрация. Оба варианта предсказывают, в какой степени пользователь будет интересоваться объектами, которые до сих пор не были им оценены. *User-based* фильтрация идентифицирует k ближайших соседей активного пользователя и на основе этих ближайших соседей вычисляет прогноз пользователя для определенного объекта интереса. В отличие от *user-based* фильтрации, при коллаборативной фильтрации на основе элементов для текущего объекта ищутся “соседи”, которые получили аналогичные рейтинги.

К плюсам данного подхода можно отнести теоретически высокую точность, к минусам: высокий порог входа - не зная ничего об интересах пользователя, рекомендации практически бесполезны.

Рекомендательные системы, основанные на контенте. Фильтрация на основе контента (англ. *Content-based filtering*) основана на предположении о том, что интересы пользователя постоянны в течение времени. Например, пользователи, заинтересованные в конкретной теме, как правило, не меняют не меняют свои интересы каждый день и будут заинтересованы в данной теме в ближайшем будущем. Фильтрация на основе контента, как показано на рисунке 1.2, в качестве входных данных содержит множество пользователей и множество категорий (или ключевых слов), которыми были помечены объекты интереса. Задача систем рекомендаций, основанных на контенте - вычислить множество объектов, которые наиболее близки к категориям, которыми интересуется текущий пользователь (P_a). Основным подходом к фильтрации на основе содержимого является сравнение уже просмотренных пользователем объектов с новыми объектами, которые потенциально могут быть рекомендованы пользователю. Базовым механизмом для определения такого сходства является извлечение ключевых слов непосредственно из контекста, содержащего объект интереса или из метаданных, которой был проаннотирован объект информации. Подобные подходы к извлечению ключевых слов рассматриваются в работе.



Рисунок 1.2 - Фильтрация на основе контента

Плюсами данного подхода является то, что можно давать рекомендации даже незнакомым пользователям, тем самым вовлекая их в сервис, появляется возможность рекомендовать те объекты, которые еще не были никем оценены.

К минусам можно отнести более низкую точность, возросшую скорость разработки.

Рекомендательные системы, основанные на знаниях. По сравнению с подходами, основанными на коллаборативной фильтрации и фильтрации на основе контента, рекомендации, основанные на знаниях (англ. *knowledge-based recommender systems*), в основном не зависят от оценки объектов или их описания с помощью метаданных, а на более глубоких правилах для выявления объектов интереса. Иногда предыдущий подход (*content-based*) определяют как частный случай *knowledge-based*, где в качестве знаний выступает информация об объектах интереса, но из-за большой распространенности систем на основе контента последние обычно выносят в отдельный тип. Дополнительные знания позволяют рекомендовать объекты, не полагаясь на «похожесть» чего-либо, а использовать более сложные условия. Рекомендации, основанная на знаниях, как показано на рисунке 1.3, опираются на следующие входные данные: (а) множество правил (ограничений) или метрик схожести и (б) множество объектов интереса. В зависимости от заданных требований пользователя, правила описывают, какие объекты должны быть рекомендованы. Текущий пользователь P_a формулирует свои предпочтения в терминах свойств элемента, которые, в свою очередь, представляются с точки зрения правил (ограничений).



Рисунок 1.3 - Рекомендации, основанные на знаниях

В качестве плюсов можно отметить возможность исключения рекомендаций уже не актуальных для данного пользователя объектов, минусы - высокая сложность построения и сбора данных.

Гибридные рекомендательные системы. Помимо трех основных подходов к фильтрации, используется гибридный подход (англ. *hybrid recommender system*), который объединяет возможности базовых типов. Использование гибридных алгоритмов позволяет достичь более высокой точности. Применяют различные стратегии построения гибридных классификаторов:

1) Взвешенная стратегия

Спрогнозированная оценка для объекта рассчитывается, как взвешенное среднее арифметическое оценок, спрогнозированных различными алгоритмами.

2) Смешанная стратегия

Смешанная гибридная стратегия основана на идее, что прогнозы отдельных рекомендации отображаются в одном интегрированном результате.

3) Каскадная стратегия

Каскадная стратегия является итеративным методом построения рекомендательных систем. Первый алгоритм играет роль грубого фильтра, а все следующие алгоритмы корректируют оценки.

2 ВЕРОЯТНОСТНАЯ МАТРИЧНАЯ ФАКТОРИЗАЦИЯ

2.1 Общие характеристики метода вероятностной матричной факторизации

Многие существующие подходы к совместной фильтрации не могут ни обрабатывать очень большие наборы данных, ни легко работать с пользователями, у которых очень мало оценок. В этом пункте представлена модель вероятностной матричной факторизации (PMF), которая линейно масштабируется в зависимости от количества наблюдений и, что более важно, хорошо работает с большим, разреженным и очень несбалансированным набором данных *Netflix*. Мы дополнительно расширяем модель PMF, включив адаптивный априори параметров модели, и показываем, как емкость модели можно регулировать автоматически. Наконец, мы представляем ограниченную версию модели PMF, основанную на предположении, что пользователи, которые оценили похожие наборы фильмов, вероятно, будут иметь схожие предпочтения. Полученная модель способна значительно лучше обобщать для пользователей с очень небольшим количеством оценок.

Один из самых популярных подходов к коллаборативной фильтрации основан на малоразмерных факторных моделях. Идея, стоящая за такими моделями, заключается в том, что отношение или предпочтения пользователя определяются небольшим количеством ненаблюдаемых факторов. В линейной факторной модели предпочтения пользователя моделируются путем линейного объединения векторов коэффициентов элементов с использованием коэффициентов, специфичных для пользователя. Например, для N пользователей и M фильмов, $N \times M$ матрица предпочтений R задается произведением $N \times D$ матрицы пользовательских коэффициентов U^T и $D \times M$ матрицы коэффициентов V . Обучение такой модели сводится к поиску наилучшего D рангового приближения к наблюдаемой целевой $N \times M$ матрице R при заданной функции потерь.

Недавно было предложено множество вероятностных моделей. Все эти модели можно рассматривать как графические модели, в которых скрытые

факторные переменные имеют прямые связи с переменными, представляющими оценки пользователей. Основным недостатком таких моделей является то, что точный вывод трудно разрешим, что означает, что для вычисления апостериорного распределения по скрытым факторам в таких моделях требуются потенциально медленные или неточные аппроксимации.

Приближения низкого ранга, основанные на минимизации расстояния в квадрате суммы, могут быть найдены с помощью разложения по сингулярным значениям (SVD). SVD находит матрицу $\hat{R} = U^T V$ данного ранга, который минимизирует расстояние в квадрате суммы до целевой матрицы R . Поскольку большинство наборов данных реального мира разрежены, большинство записей в R будут отсутствовать. В этих случаях расстояние в квадрате суммы вычисляется только для наблюдаемых элементов целевой матрицы R . Эта, казалось бы, незначительная модификация приводит к сложной невыпуклой задаче оптимизации, которая не может быть решена с использованием стандартных реализаций SVD.

Вместо ограничения ранга матрицы аппроксимации $\hat{R} = U^T V$, т.е. количества факторов, можно штрафовать нормы U и V . Обучение в этой модели, однако, требует решения разреженной полуопределенной программы (SDP), что делает этот подход неосуществимым для наборов данных, содержащих миллионы наблюдений.

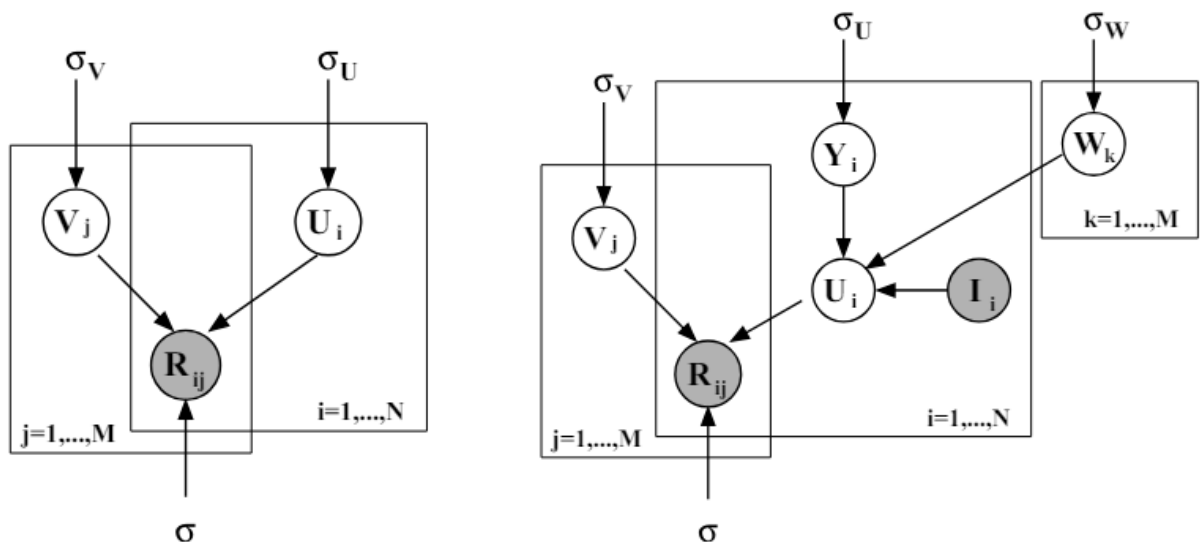


Рисунок 2.1 - На левой панели показана графическая модель для вероятностной матричной факторизации (PMF). Правая панель показывает графическую модель для ограниченного PMF.

Многие из упомянутых выше алгоритмов совместной фильтрации были применены для моделирования пользовательских рейтингов в наборе данных *Netflix Prize*, который содержит 480 189 пользователей, 17 770 фильмов и более 100 миллионов наблюдений (пользователь / фильм / рейтинг утраивается). Однако ни один из этих методов не оказался особенно успешным по двум причинам. Во-первых, ни один из вышеупомянутых подходов, за исключением основанных на матричной факторизации, хорошо масштабируется для больших наборов данных. Во-вторых, большинство существующих алгоритмам сложно делать точные прогнозы для пользователей, у которых очень мало оценок.

2.2 Вероятностная матричная факторизация (PMF)

Предположим, у нас есть M фильмов, N пользователей и целые значения рейтинга от 1 до K . Пусть R_{ij} представляет рейтинг пользователя i для просмотра фильмов j , $U \in \mathbb{R}^{D \times N}$ и $V \in \mathbb{R}^{D \times M}$ матрицы скрытых характеристик пользователей и фильмов, при этом вектор-столбец U_i и вектор-столбец V_j представляют векторы скрытых характеристик для конкретного пользователя и конкретного фильма соответственно. Поскольку производительность модели измеряется путем вычисления среднеквадратичной ошибки (RMSE) на тестовом наборе, мы сначала используем вероятностную линейную модель с гауссовым шумом наблюдения, как показано на рисунке 2.1. Мы определяем условное распределение по наблюдаемым оценкам как

$$p(R | U, V, \sigma^2) = \prod_{i=1}^N \prod_{j=1}^M [\mathcal{N}(R_{ij} | U_i^T V_j, \sigma^2)]^{I_{ij}}, \quad (2.1)$$

где $\mathcal{N}(x | \mu, \sigma^2)$ - функция плотности вероятности гауссова распределения со средним значением μ и дисперсией σ^2 , а I_{ij} - индикаторная функция, которая равна 1, если пользователь i оценил фильм j , и равна 0 в противном случае. Мы также устанавливаем сферические гауссовы априорные значения с

нулевым средним значением [1, 11] для векторов характеристик пользователя и фильма:

$$p(U | \sigma_U^2) = \prod_{i=1}^N \mathcal{N}(U_i | 0, \sigma_U^2 \mathbf{I}), \quad p(V | \sigma_V^2) = \prod_{j=1}^M \mathcal{N}(V_j | 0, \sigma_V^2 \mathbf{I}), \quad (2.2)$$

Журнал последующего распространения по функциям пользователя и фильма:

$$\begin{aligned} \ln p(U, V | R, \sigma^2, \sigma_V^2, \sigma_U^2) = & -\frac{1}{2\sigma^2} \sum_{i=1}^N \sum_{j=1}^M I_{ij} (R_{ij} - U_i^T V_j)^2 - \frac{1}{2\sigma_U^2} \sum_{i=1}^N U_i^T U_i - \\ & - \frac{1}{2\sigma_V^2} \sum_{j=1}^M V_j^T V_j - \frac{1}{2} \left(\left(\sum_{i=1}^N \sum_{j=1}^M I_{ij} \right) \ln \sigma^2 + N D \ln \sigma_U^2 + M D \ln \sigma_V^2 \right) + C, \end{aligned} \quad (2.3)$$

где C - это константа, которая не зависит от параметров. Максимизация логарифмической зависимости от видео и пользовательских функций при сохранении фиксированных гиперпараметров (т.е. дисперсии шума наблюдения и предшествующих дисперсий) эквивалентна минимизации целевой функции суммы квадратов ошибок с использованием квадратичных условий регуляризации:

$$E = \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^M I_{ij} (R_{ij} - U_i^T V_j)^2 + \frac{\lambda_U}{2} \sum_{i=1}^N \|U_i\|_{Fro}^2 + \frac{\lambda_V}{2} \sum_{j=1}^M \|V_j\|_{Fro}^2, \quad (2.4)$$

где $\lambda_U = \sigma^2 / 2\sigma_U^2$, $\lambda_V = \sigma^2 / 2\sigma_V^2$, и $\|\cdot\|_{Fro}^2$ обозначает норму Фробениуса. Локальный минимум целевой функции, заданный уравнением 2.4, можно найти, выполнив градиентный спуск в U и V . Обратите внимание, что эту модель можно рассматривать как вероятностное расширение модели SVD, поскольку, если все оценки соблюдены, цель, заданная уравнением 2.4 сводится к цели SVD в пределе предшествующих отклонений, уходящих в бесконечность.

В наших экспериментах вместо использования простой линейно-гауссовой модели, которая может делать прогнозы за пределами диапазона допустимых значений рейтинга, точечное произведение векторов

характеристик, относящихся к пользователю и фильму, передается через логистическую функцию $g(x) = 1/(1 + \exp(-x))$, который ограничивает диапазон прогнозов:

$$p(R | U, V, \sigma^2) = \prod_{i=1}^N \prod_{j=1}^M [\mathcal{N}(R_{ij} | g(U_i^T V_j), \sigma^2)]^{I_{ij}} \quad (2.5)$$

Мы сопоставляем рейтинги 1, ..., K с интервалом [0, 1], используя функцию $t(x) = (x - 1)/(K - 1)$, чтобы диапазон допустимых значений рейтинга соответствовал диапазону прогнозов, которые делает наша модель. Минимизация целевой функции, приведенной выше, с использованием метода наискорейшего спуска, требует времени, линейного по количеству наблюдений.

2.3 Реализация модели PMF на python

Начнём с определения векторов скрытых характеристик пользователей и фильмов соответствующим оценкам. Опишем алгоритм только для скрытых характеристик пользователей, а для скрытых характеристик алгоритм будет аналогичен. На рисунке 2.2 сначала из списка data берётся нулевой столбец – это все индексы пользователей, соответствующих оценкам, а затем берутся векторы скрытых характеристик пользователей по этим индексам. В начале эти вектора задаются случайным образом.

```
u_features = self.user_features_.take(
    data.take(0, axis=1), axis=0)
i_features = self.item_features_.take(
    data.take(1, axis=1), axis=0)
```

Рисунок 2.2 – Определение векторов скрытых характеристик соответствующим оценкам

Получается список векторов k мерного пространства, где количество векторов – это количество оценок, а k – это количество скрытых характеристик.

```

preds = np.sum(u_features * i_features, 1)
errs = preds - (data.take(2, axis=1) - self.mean_rating_)
err_mat = np.tile(2 * errs, (self.n_feature, 1)).T
u_grads = i_features * err_mat + self.reg * u_features
i_grads = u_features * err_mat + self.reg * i_features

u_feature_grads.fill(0.0)
i_feature_grads.fill(0.0)
for i in xrange(data.shape[0]):
    row = data.take(i, axis=0)
    u_feature_grads[row[0], :] += u_grads.take(i, axis=0)
    i_feature_grads[row[1], :] += i_grads.take(i, axis=0)

```

Рисунок 2.3 – Градиентный спуск в U и V

На рисунке 2.3 мы ищем локальный минимум целевой функции, заданный уравнением 2.4, выполнив наискорейший градиентный спуск в U и V. У нас есть известные оценки и предсказанные оценки (произведение матрицы коэффициентов U для пользователей и матрицы коэффициентов V для фильмов). Здесь self.reg - это параметр регуляризации который равен 0.01.

```

u_feature_mom = (self.momentum * u_feature_mom) + \
    ((self.epsilon / data.shape[0]) * u_feature_gradients)
i_feature_mom = (self.momentum * i_feature_mom) + \
    ((self.epsilon / data.shape[0]) * i_feature_gradients)

self.user_features_ -= u_feature_mom
self.item_features_ -= i_feature_mom

```

Рисунок 2.4 – Обновление импульса и векторов скрытых характеристик пользователей и фильмов

Здесь происходит обновление величины, с которой изменятся наши латентные вектора, и соответственно их обновление. Скорость обучения задаётся константой self.momentum и равна 0.8. После обновления латентных векторов итерация заканчивается, и если условие выхода не было выполнено, начинается новая итерация.

2.4 Экспериментальные результаты

На рисунке 2.5 изображены результаты обучения модели PMF. Значения RMSE на первых итерациях модели были высоки. И хотя модель значительно

уменьшила RMSE во время обучения её значение всё ещё достаточно большое (0.91).

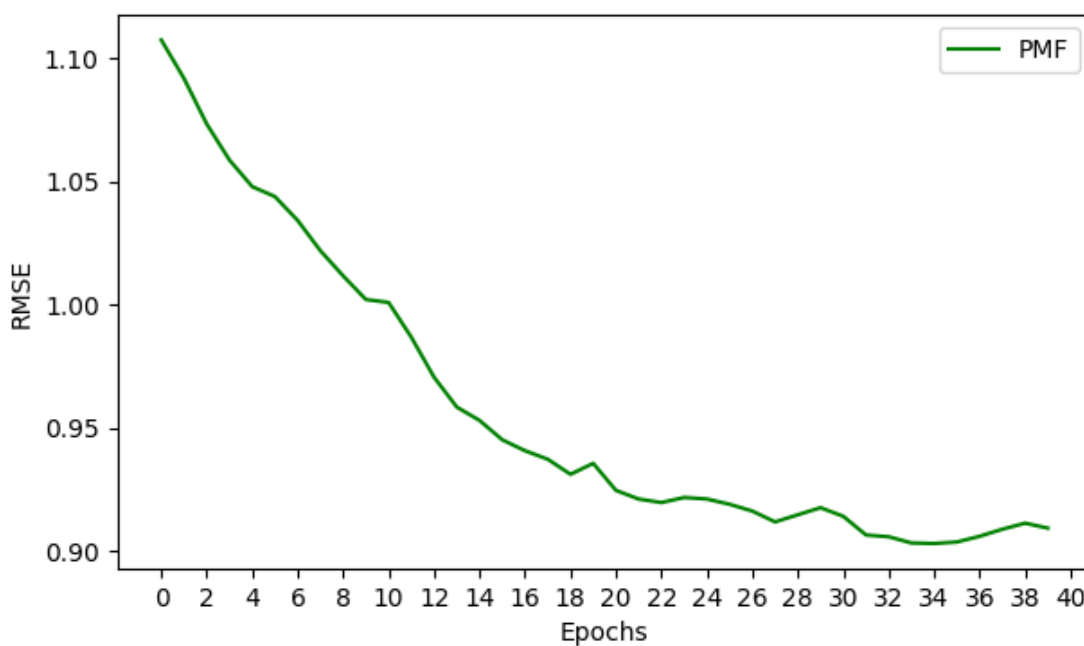


Рисунок 2.5 – Результаты обучения модели PMF

Результаты обучения соответствуют ожиданиям, а значит реализация модели успешен.

3 БАЙЕСОВСКАЯ ВЕРОЯТНОСТНАЯ МАТРИЧНАЯ ФАКТОРИЗАЦИЯ

3.1 Общие характеристики метода Байесовской вероятностной матричной факторизации

Методы аппроксимации матриц низкого ранга представляют собой один из самых простых и эффективных подходов к коллаборативной фильтрации. Такие модели обычно адаптируются к данным путем нахождения картографической оценки параметров модели, что может быть эффективно выполнено даже на очень больших наборах данных. Однако, если параметры регуляризации не настроены тщательно, этот подход склонен к переобучению, поскольку он позволяет получить оценку параметров в одной точке. В этой главе мы представляем полностью байесовскую трактовку модели вероятностной матричной факторизации (PMF), в которой производительность модели контролируется автоматически путем интегрирования по всем параметрам модели и гиперпараметрам. Мы показываем, что байесовские модели PMF могут быть эффективно обучены с использованием методов Монте-Карло с цепочкой Маркова.

Факторные модели широко используются в области коллаборативной фильтрации для моделирования пользовательских предпочтений. Идея, лежащая в основе таких моделей, заключается в том, что предпочтения пользователя определяются небольшим количеством ненаблюдаемых факторов. В линейной факторной модели оценка товара пользователем моделируется с помощью внутреннего произведения вектора факторов товара и вектора факторов пользователя. Это означает, что матрица предпочтений $N \times M$ оценок, которые N пользователей присваивают M фильмам, моделируется произведением матрицы коэффициентов U для пользователей $D \times N$ и матрицы коэффициентов V для фильмов $D \times M$. Обучение такой модели сводится к нахождению наилучшего приближения ранга D к наблюдаемой целевой $N \times M$ матрице R при заданной функции потерь.

Было предложено множество вероятностных моделей, основанных на факторах. В этих моделях факторные переменные считаются незначительно независимыми, в то время как рейтинговые переменные считаются условно независимыми с учетом факторных переменных. Основным недостатком таких моделей является то, что вывод апостериорного распределения по факторам с учетом рейтингов является трудноразрешимой задачей. Многие из существующих методов основаны на выполнении картографической оценки параметров модели. Обучение таких моделей сводится к максимизации логарифмической зависимости от параметров модели и может быть выполнено очень эффективно даже на очень больших наборах данных.

На практике мы обычно заинтересованы в прогнозировании рейтингов для новых пар пользователь/фильм, а не в оценке параметров модели. Эта точка зрения предполагает использование байесовского подхода к проблеме, который предполагает интеграцию параметров модели. В этой главе мы описываем полностью байесовскую трактовку модели вероятностной матричной факторизации (PMF), которая недавно была применена для коллаборативной фильтрации. Отличительной особенностью этой работы является использование методов Монте-Карло с цепью Маркова (MCMC) для приближенного вывода в этой модели. На практике методы MCMC редко используются для решения крупномасштабных задач, поскольку практикующие специалисты считают их очень медленными.

Предыдущие применения методов байесовской матричной факторизации для коллаборативной фильтрации использовали вариационную аппроксимацию для выполнения логического вывода. Эти методы пытаются аппроксимировать истинное апостериорное распределение более простым факторизованным распределением, при котором векторы пользовательских факторов не зависят от векторов факторов фильма. Следствием этого предположения является то, что вариационные апостериорные распределения по факторным векторам являются произведением двух многомерных гауссиан: одного для векторов факторов просмотра и одного для векторов

факторов просмотра фильмов. Это предположение о независимости факторов, влияющих на зрителя, от факторов, влияющих на фильм, кажется необоснованным, и, как показывают наши эксперименты, распределения по факторам в таких моделях оказываются негауссовыми.

3.2 Модель байесовской вероятностной матричной факторизации

Графическая модель, представляющая байесовскую PMF, показана на рисунке 3.1 (правая панель). Как и в PMF, вероятность наблюдаемых оценок определяется уравнением 3.1. Предполагается, что предыдущие распределения по векторам характеристик пользователя и фильма являются гауссовыми:

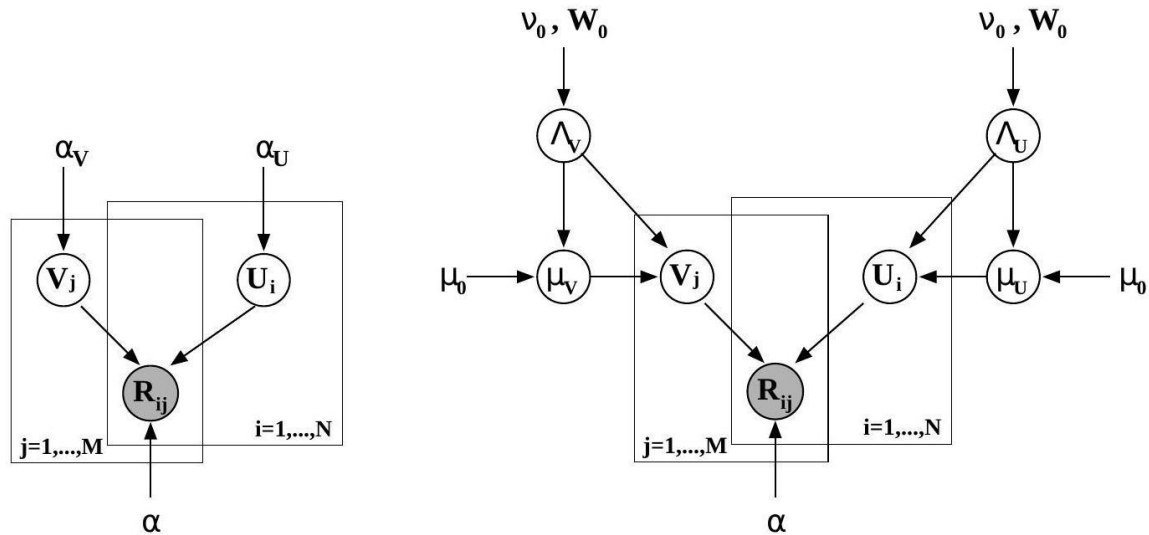


Рисунок 3.1 - На левой панели показана графическая модель для вероятностной матричной факторизации (PMF). На правой панели показана графическая модель для байесовской PMF.

$$p(U \mid \mu_U, \Lambda_U) = \prod_{i=1}^N \mathcal{N}(U_i \mid \mu_U, \Lambda_U^{-1}), \quad (3.1)$$

$$p(V \mid \mu_V, \Lambda_V) = \prod_{i=1}^M \mathcal{N}(V_i \mid \mu_V, \Lambda_V^{-1}). \quad (3.2)$$

Далее мы применяем критерии Гаусса-Уишарта к гиперпараметрам пользователя и фильма $\Theta_U = \{\mu_U, \Lambda_U\}$ и $\Theta_V = \{\mu_V, \Lambda_V\}$:

$$p(\Theta_U | \Theta_0) = p(\mu_U | \Lambda_U)p(\Lambda_U) = \mathcal{N}(\mu_U | \mu_0, (\beta \Lambda_U)^{-1})\mathcal{W}(\Lambda_U | W_0, \nu_0), \quad (3.3)$$

$$p(\Theta_V | \Theta_0) = p(\mu_V | \Lambda_V)p(\Lambda_V) = \mathcal{N}(\mu_V | \mu_0, (\beta \Lambda_V)^{-1})\mathcal{W}(\Lambda_V | W_0, \nu_0). \quad (3.4)$$

Здесь $\mathcal{W}$ - распределение Уишарта с ν_0 степенями свободы и масштабной $D \times D$ матрицей W_0 :

$$\mathcal{W}(\Lambda | W_0, \nu_0) = \frac{1}{C} |\Lambda|^{\frac{(\nu_0 - D - 1)}{2}} \exp\left(-\frac{1}{2} \text{Tr}(W_0^{-1} \Lambda)\right) \quad (3.5)$$

где C нормализующая константа. Для удобства мы также определяем $\Theta_0 = \{\mu_0, \nu_0, W_0\}$. В наших экспериментах мы также устанавливаем $\nu_0 = D$ и W_0 в качестве идентификационной матрицы как для гиперпараметров пользователя, так и для гиперпараметров фильма и выбираем $\mu_0 = 0$ по симметрии.

3.3 Прогнозы

Прогнозируемое распределение рейтингового значения R_{ij}^* для пользователя i и фильма с запросом j получено путем маргинализации по параметрам модели и гиперпараметрам:

$$p(R_{ij}^* | R, \Theta_0) = \iint p(R_{ij}^* | U_i, V_j) p(U, V | R, \Theta_U, \Theta_V) \quad (3.6)$$

Поскольку точная оценка этого прогнозирующего распределения аналитически трудноразрешима из-за сложности апостериорных данных, нам необходимо прибегнуть к приближительному выводу.

Одним из вариантов было бы использование вариационных методов, которые обеспечивают детерминированные схемы аппроксимации для последующих элементов. В частности, мы могли бы аппроксимировать истинное апостериорное значение $p(U, V, \Theta_U, \Theta_V | R)$ распределением, которое умножает, причем каждый множитель имеет определенную параметрическую форму, такую как гауссово распределение. Это приближенное апостериорное значение позволило бы нам аппроксимировать интегралы в уравнении 3.11. Вариационные методы стали предпочтительной методологией, поскольку они обычно хорошо масштабируются для больших приложений. Однако они могут давать неточные результаты, поскольку, как

правило, используют слишком простые аппроксимации апостериорных значений.

Методы, основанные на МСМС, с другой стороны, используют аппроксимацию методом Монте-Карло для прогнозирующего распределения для уравнения 3.11:

$$p(R_{ij}^* | R, \Theta_0) \approx \frac{1}{K} \sum_{k=1}^K p(R_{ij}^* | U_i^{(k)}, V_j^{(k)}). \quad (3.7)$$

Выборки $\{U_i^{(k)}, V_j^{(k)}\}$ генерируются путем запуска цепи Маркова, стационарное распределение которой является апостериорным распределением по параметрам модели и гиперпараметрам $\{U, V, \Theta_U, \Theta_V\}$. Преимущество методов, основанных на методе Монте-Карло, заключается в том, что асимптотически они дают точные результаты. Однако на практике методы МСМС обычно воспринимаются как настолько сложные с точки зрения вычислений, что их использование ограничивается маломасштабными задачами.

3.4 Алгоритм МСМС

Одним из простейших алгоритмов МСМС является алгоритм выборки Гиббса, который циклически перебирает скрытые переменные, отбирая каждую из них из ее распределения в зависимости от текущих значений всех других переменных. Выборка Гиббса обычно используется, когда эти условные распределения могут быть легко выбраны.

Благодаря использованию сопряженных априорных значений для параметров и гиперпараметров в байесовской модели PMF, условные распределения, полученные на основе апостериорного распределения, легко поддаются выборке. В частности, условное распределение по вектору пользовательских характеристик U_i , обусловленное особенностями фильма, наблюдаемой матрицей рейтингов пользователей R , и значениями гиперпараметров, является гауссовым:

$$p(U_i | R, V, \Theta_U, \alpha) = \mathcal{N}(U_i | \mu_i^*, [\Lambda_i^*]^{-1})$$

$$\sim \prod_{j=1}^M [\mathcal{N}(R_{ij} | U_i^T V_j, \alpha^{-1})]^{I_{ij}} p(U_i | \mu_U, \Lambda_U) \quad (3.8)$$

где

$$\Lambda_i^* = \Lambda_U + \alpha \sum_{j=1}^M [V_j V_j^T]^{I_{ij}} \quad (3.9)$$

$$\mu_i^* = [\Lambda_i^*]^{-1} \left(\alpha \sum_{j=1}^M [V_j R_{ij}]^{I_{ij}} + \Lambda_U \mu_U \right). \quad (3.10)$$

Обратите внимание, что условное распределение по матрице скрытых признаков пользователя U преобразуется в произведение условных распределений по отдельным векторам признаков пользователя:

$$p(U | R, V, \Theta_U) = \prod_{i=1}^N p(U_i | R, V, \Theta_U) \quad (3.11)$$

Поэтому мы можем легко ускорить выборку, используя параллельную выборку из этих условных распределений. Ускорение может быть существенным, особенно при большом количестве пользователей.

Условное распределение по пользовательским гиперпараметрам, обусловленное матрицей U пользовательских признаков, задается распределением Гаусса-Уишарта:

$$p(\mu_U, \Lambda_U | U, \Theta_0) = \mathcal{N}(\mu_U | \mu_0^*, (\beta \Lambda_U)^{-1}) \mathcal{W}(\Lambda_U | W_0^* \nu_0^*) \quad (3.12)$$

где

$$\mu_0^* = \frac{\beta_0 \mu_0 + N \bar{U}}{\beta_0 + N}, \quad \beta_0^* = \beta_0 + N, \quad \nu_0^* = \nu_0 + N$$

$$[W_0^*]^{-1} = W_0^{-1} + N \bar{S} + \frac{\beta_0 N}{\beta_0 + N} (\mu_0 - \bar{U})(\mu_0 - \bar{U})^T \quad (3.13)$$

$$\bar{U} = \frac{1}{N} \sum_{i=1}^N U_i \quad \bar{S} = \frac{1}{N} \sum_{i=1}^N U_i U_i^T$$

Условные распределения по векторам характеристик фильма и гиперпараметрам фильма имеют точно такой же вид. Алгоритм выборки Гиббса в этом случае принимает следующий вид:

- 1) Инициализируем параметров модели $\{U^1, V^1\}$
- 2) Для $t = 1, \dots, T$

а) Производим выборку гиперпараметров (уравнение 3.11):

$$\Theta_U^t \sim p(\Theta_U | U^t, \Theta_0) \quad (3.14)$$

$$\Theta_V^t \sim p(\Theta_V | V^t, \Theta_0) \quad (3.15)$$

б) Для каждого $i = 1, \dots, N$ производим выборку характеристик пользователя (уравнение 3.8):

$$U_i^{t+1} \sim p(U_i | R, V^t, \Theta_U^t) \quad (3.16)$$

в) Для каждого $i = 1, \dots, M$ производим выборку характеристик фильма:

$$V_i^{t+1} \sim p(V_i | R, U^{t+1}, \Theta_V^t) \quad (3.17)$$

3.5 Реализация модели BP MF на python

Сначала происходит инициализация параметров модели $\{U^1, V^1\}$. Затем начинается главный цикл. На рисунке 3.2 показана итерация MCMC. Сначала производится выборка гиперпараметров, затем для каждого производится выборка характеристик пользователя и характеристик фильма.

```
self._update_item_params()
self._update_user_params()

self._update_item_features()
self._update_user_features()

self._update_average_features(self.iter_)
```

Рисунок 3.2 – Главный цикл MCMC

На рисунке 3.3 показан алгоритм выборки гиперпараметров.

```

def _update_user_params(self):
    N = self.n_user
    X_bar = np.mean(self.user_features_, 0).reshape((self.n_feature, 1))
    S_bar = np.cov(self.user_features_.T)

    diff_X_bar = self.mu0_user - X_bar

    Wishart_post = inv(inv(self.Wishart_user) +
                       N * S_bar +
                       np.dot(diff_X_bar, diff_X_bar.T) *
                       (N * self.beta_user) / (self.beta_user + N))
    Wishart_post = (Wishart_post + Wishart_post.T) / 2.0

    df_post = self.df_user + N
    self.alpha_user = wishart.rvs(df_post, Wishart_post, 1, self.random_state)

    mu_mean = (self.beta_user * self.mu0_user + N * X_bar) / \
              (self.beta_user + N)

    mu_var = cholesky(inv(np.dot(self.beta_user + N, self.alpha_user)))
    self.mu_user = mu_mean + np.dot(
        mu_var, self.random_state.randn(self.n_feature, 1))

```

Рисунок 3.3 – Выборка гиперпараметров

На рисунке 3.3 предоставлена реализация формулы 3.8. Здесь мы применяем критерии Гаусса-Уишарта к гиперпараметрам пользователя.

```

def _update_user_features(self):
    for user_id in xrange(self.n_user):
        indices = self.ratings_csr[user_id, :].indices
        features = self.item_features[indices, :]
        rating = self.ratings_csr[user_id, :].data - self.mean_rating_
        rating = np.reshape(rating, (rating.shape[0], 1))

        covar = inv(
            self.alpha_user + self.beta * np.dot(features.T, features))
        lam = cholesky(covar)

        temp = (self.beta * np.dot(features.T, rating) +
                np.dot(self.alpha_user, self.mu_user))
        mean = np.dot(covar, temp)
        temp_feature = mean + np.dot(
            lam, self.random_state.randn(self.n_feature, 1))
        self.user_features[user_id, :] = temp_feature.ravel()

```

Рисунок 3.4 – Выборка характеристик пользователя

На рисунке 3.4 предоставлена реализация формулы 3.6. Условное распределение по вектору характеристик пользователя, обусловленное особенностями фильма.

3.6 Экспериментальные результаты

На рисунке 3.5 представлены результаты обучения модели BPMF.

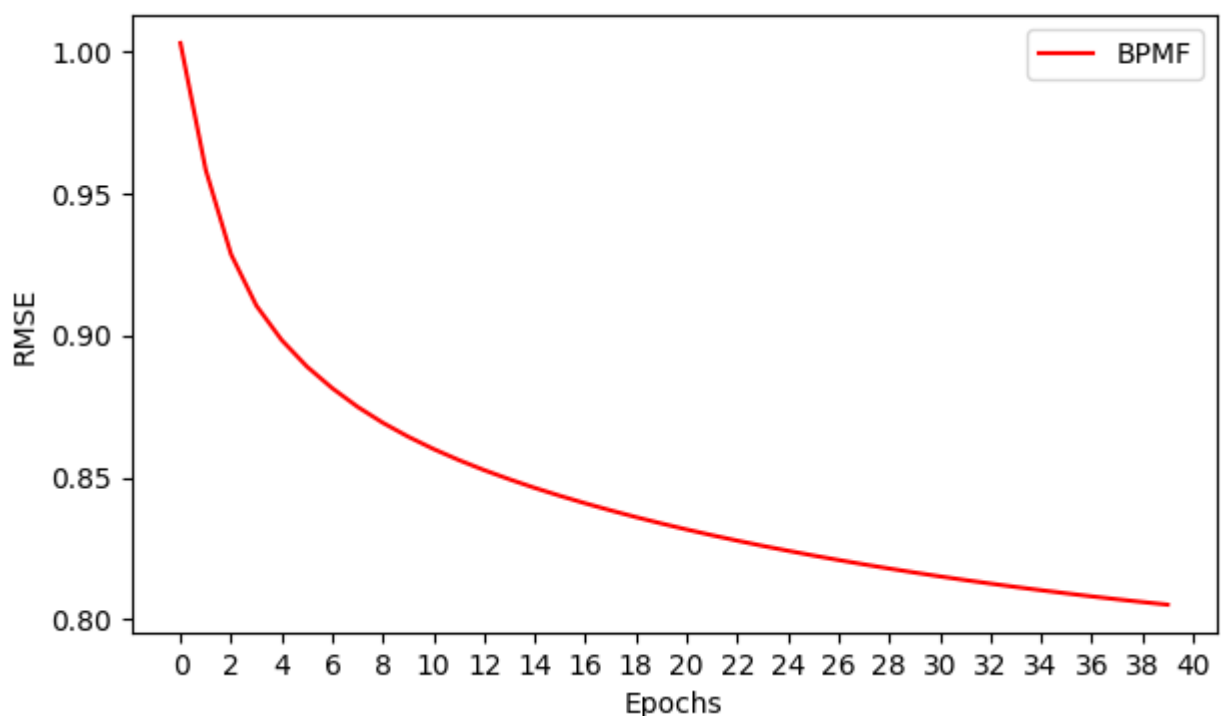


Рисунок 3.5 – Результаты обучения модели ВРМФ

Из рисунка видно, что модель в первые пять итераций достигла хороших результатов, а после замедлилась, но не слишком сильно. Даже в конце 40 итерации можно заметить небольшие улучшения RMSE, что значит впереди есть потенциал обучения. Модель показала хорошие результаты ($RMSE=0.81$) в соответствии с ожиданиями, а значит её реализация успешно выполнена.

4 КРУПНОМАСШТАБНАЯ ПАРАЛЛЕЛЬНАЯ КОЛЛАБОРАТИВНАЯ ФИЛЬТРАЦИЯ

4.1 Общие характеристики метода крупномасштабной параллельной коллаборативной фильтрации

Многие рекомендательные системы предлагают товары пользователям, используя методы коллаборативной фильтрации (CF), основанные на исторических записях о товарах, которые пользователи просматривали, покупали или оценивали. Две основные проблемы, с которыми приходится сталкиваться большинству CF-систем, - это масштабируемость и разреженность пользовательских профилей. Чтобы решить эти проблемы, в этой главе мы опишем алгоритм CF, использующий метод наименьших квадратов с взвешенной λ -регуляризацией (ALS-WR).

Пусть $R = \{r_{ij}\}_{n_u \times n_m}$ обозначает матрицу пользовательских фильмов, где каждый элемент r_{ij} представляет собой рейтинг фильма j , оцененный пользователем i причем его значение либо является действительным числом, либо отсутствует, n_u обозначает количество пользователей, а n_m указывает количество фильмов. Как и в большинстве рекомендательных систем, наша задача состоит в том, чтобы оценить некоторые недостающие значения в R на основе известных значений.

Мы начинаем с низкоранговой аппроксимации матрицы рейтингов R . Этот подход моделирует как пользователей, так и фильмы, присваивая им координаты в пространстве объектов малой размерности. У каждого пользователя и каждого фильма есть вектор характеристик, и каждая оценка (известная или неизвестная) фильма пользователем моделируется как внутреннее произведение соответствующих векторов характеристик пользователя и фильма. Более конкретно, пусть $U = [\mathbf{u}_i]$ матрица характеристик пользователя, где $\mathbf{u}_i \in \mathbb{R}^{n_f}, i = 1 \dots n_u$, обозначает i^{th} столбец в U , и пусть $M = [\mathbf{m}_j]$ матрица характеристик фильма, где $\mathbf{m}_j \in \mathbb{R}^{n_f}$ для всех $j = 1 \dots n_m$. Здесь n_f - размерность пространственных объектов, то есть

количество скрытых переменных в модели. Это системный параметр, который может быть определен с помощью резервного набора данных или перекрестной проверки. Если бы оценки пользователей были полностью предсказуемыми и n_f достаточно велико, мы могли бы ожидать, что $r_{ij} = \langle \mathbf{u}_i, \mathbf{m}_j \rangle, \forall i, j$. Однако на практике мы минимизируем функцию потерь (из U и M) чтобы получить матрицы U и M . Потери, связанные с одним рейтингом, определяются как квадрат ошибки:

$$\mathcal{L}^2(r, \mathbf{u}, \mathbf{m}) = (r - \langle \mathbf{u}, \mathbf{m} \rangle)^2. \quad (4.1)$$

Затем мы можем определить эмпирическую общую потерю (для данной пары U и M) как сумму потерь по всем известным рейтингам в уравнении (4.2).

$$\mathcal{L}^{emp}(R, U, M) = \frac{1}{n} \sum_{(i,j) \in I} \mathcal{L}^2(r_{ij}, \mathbf{u}_i, \mathbf{m}_j) \quad (4.2)$$

где I - набор индексов известных оценок, а n – размер I .

Мы можем сформулировать задачу аппроксимации низкого ранга следующим образом.

$$(U, M) = \arg \min_{(U, M)} \mathcal{L}^{emp}(R, U, M). \quad (4.3)$$

где U и M являются реальными, имеют n_f возможных значений, но в остальном не ограничены.

В этой задаче (уравнение (4.3)) необходимо определить свободные параметры $(n_u + n_m) \times n_f$. С другой стороны, известный набор рейтингов I содержит гораздо меньше элементов, чем $n_u n_m$ поскольку все, за исключением очень небольшого числа пользователей, не могут просмотреть и оценить больше 10 000 фильмов. Решение уравнения задачи (4.3) с большим количеством параметров (когда n_f относительно велико) из разреженного набора данных обычно приводит к переопределению данных. Чтобы избежать переобучения, распространенный метод добавляет термин регуляризации Тихонова к эмпирической функции риска (уравнение (4.4)).

$$\mathcal{L}_\lambda^{reg}(R, U, M) = \mathcal{L}^{emp}(R, U, M) + \lambda(\|U\Gamma_U\|^2 + \|M\Gamma_M\|^2), \quad (4.4)$$

для некоторых подходящим образом выбранных матриц Тихонова Γ_U и Γ_M . Подробности мы обсудим в следующем параграфе.

4.2 ALS со взвешенной λ -регуляризацией

Поскольку рейтинговая матрица содержит как сигналы, так и шум, важно удалить шум и использовать восстановленный сигнал для прогнозирования пропущенных оценок. Сингулярное значение Декомпозиции (SVD) - это естественный подход, который аппроксимирует исходную пользовательская матрица рейтинга фильмов R произведением двух рейтинговых матриц $R = U^T \times M$. Решение, полученное с помощью SVD, минимизирует норму Фробениуса $R-R$, что эквивалентно минимизации RMSE по всем элементам R . Однако, поскольку в рейтинговой матрице R много недостающих элементов, стандартные алгоритмы SVD не могут найти U и M .

В этой главе мы используем чередующиеся наименьшие квадраты (ALS) для решения задачи разложения матрицы низкого ранга на множители следующим образом:

Шаг 1 Инициализируйте матрицу M , назначив среднюю оценку для этого фильма в качестве первой строки и небольшие случайные числа для остальных записей;

Шаг 2 Исправьте M , решите U , минимизировав целевую функцию (сумму квадратов ошибок);

Шаг 3 Исправьте U , решите M , минимизировав целевую функцию аналогичным образом;

Шаг 4 Повторяйте шаги 2 и 3 до тех пор, пока не будет удовлетворен критерий остановки.

Используемый нами критерий остановки основан на наблюдаемом RMSE в наборе данных зонда. После одного раунда обновления как U , так и M , если разница между наблюдаемыми значениями RMSE в тестовом наборе данных составляет менее 0.0001, итерация останавливается, и мы используем полученные значения U , M для составления окончательных прогнозов по тестовому набору данных.

Как мы упоминали в разделе 2, существует множество свободных параметров. Без регуляризации ALS может привести к переобучению. Распространенным решением является использование регуляризации по Тихонову, которая ограничивает использование больших параметров. Мы перепробовали различные матрицы регуляризации и в итоге пришли к выводу, что следующая взвешенная λ -регуляризация работает лучше всего, поскольку она никогда не переоценивает тестовые данные (эмпирически), когда мы увеличиваем количество функций или итераций.

$$f(U, M) = \sum_{(i,j) \in I} (r_{ij} - \mathbf{u}_i^T \mathbf{m}_j)^2 + \lambda \left(\sum_i n_{u_i} \|\mathbf{u}_i\|^2 + \sum_j n_{m_j} \|\mathbf{m}_j\|^2 \right), \quad (4.5)$$

где n_{u_i} и n_{m_j} обозначают количество оценок пользователя i и фильма j соответственно. Пусть I_i обозначает набор фильмов j которые оценил пользователь i тогда n_{u_i} - это мощность I_i ; аналогично I_j обозначает набор пользователей, которые оценили фильм j , а n_{m_j} - мощность I_j . Это соответствует регуляризации Тихонова, где $\Gamma_U = \text{diag}(n_{u_i})$ и $\Gamma_M = \text{diag}(n_{m_j})$.

Теперь мы покажем, как решить матрицу U , когда M задано. Заданный столбец U , скажем, u_i , определяется путем решения регуляризованной линейной задачи наименьших квадратов, включающей известные рейтинги пользователя i , и векторы характеристик m_j фильмов, которые оценил пользователь i .

$$\begin{aligned} & \frac{1}{2} \frac{\partial f}{\partial u_{ki}} = 0, \forall i, k \\ \Rightarrow & \sum_{j \in I_i} (\mathbf{u}_i^T \mathbf{m}_j - r_{ij}) m_{kj} + \lambda n_{u_i} u_{ki} = 0, \forall i, k \\ \Rightarrow & \sum_{j \in I_i} m_{kj} \mathbf{m}_j^T \mathbf{u}_i + \lambda n_{u_i} u_{ki} = \sum_{j \in I_i} m_{kj} r_{ij}, \forall i, k \\ \Rightarrow & (M_{I_i} M_{I_i}^T + \lambda n_{u_i} E) \mathbf{u}_i = M_{I_i} R^T(i, I_i), \forall i \\ \Rightarrow & \mathbf{u}_i = A_i^{-1} V_i, \forall i, \end{aligned} \quad (4.6)$$

где $A_i = M_{I_i} M_{I_i}^T + \lambda n_{u_i} E$, $V_i = M_{I_i} R^T(i, I_i)$, а E - это единичная матрица $n_f \times n_f$. M_{I_i} обозначает подматрицу из M , где столбцы $j \in I_i$ выбираются, а $R(i, I_i)$ - это вектор строк, в котором берутся столбцы $j \in I_i$ из i -й строки R .

Аналогично, когда M обновляется, мы можем вычислить отдельные значения m_j с помощью регуляризованного линейного метода наименьших квадратов, используя векторы характеристик пользователей, которые оценили фильм j , и их оценки его следующим образом:

$$\mathbf{m}_j = A_j^{-1} V_j, \forall j, \quad (4.7)$$

где $A_j = U_{I_j} U_{I_j}^T + \lambda n_{m_j} E$ и $V_j = U_{I_j} R(I_j, j)$. U_{I_j} обозначает подматрицу U , в которой выбраны столбцы $i \in I_j$, а $R(I_j, j)$ - это вектор-столбец, в котором берутся строки $i \in I_j$ j -го столбца из R .

4.3 Реализация модели als на python

В главном цикле итераций, как показано на рисунке 4.1, есть только две основные функции – обновление признаков пользователя и обновления признаков фильма. Сначала идёт обновление признаков пользователя с помощью исправления M , решения U , минимизирования целевой функции (суммы квадратов ошибок). В следующей функции – наоборот.

```
self._update_user_feature()
self._update_item_feature()
```

Рисунок 4.1 - Главный цикл итераций

На рисунке 4.2 предоставлена реализация формулы 4.6.

```

for i in xrange(self.n_user):
    _, item_idx = self.ratings_csr[i, :].nonzero()
    n_u = item_idx.shape[0]
    if n_u == 0:
        logger.debug("no ratings for user %d", i)
        continue
    item_features = self.item_features_.take(item_idx, axis=0)
    ratings = self.ratings_csr[i, :].data - self.mean_rating_

    A_i = (np.dot(item_features.T, item_features) +
           self.reg * n_u * np.eye(self.n_feature))
    V_i = np.dot(item_features.T, ratings)
    self.user_features_[i, :] = np.dot(inv(A_i), V_i)

```

Рисунок 4.2 – Обновление признаков пользователя

Здесь используется регуляризация по Тихонову, которая представлена в формуле 4.5. После выполнения обновления признаков пользователей и фильмов, цикл повторяется.

4.4 Экспериментальные результаты

На рисунке 4.3 представлены результаты обучения модели ALS.

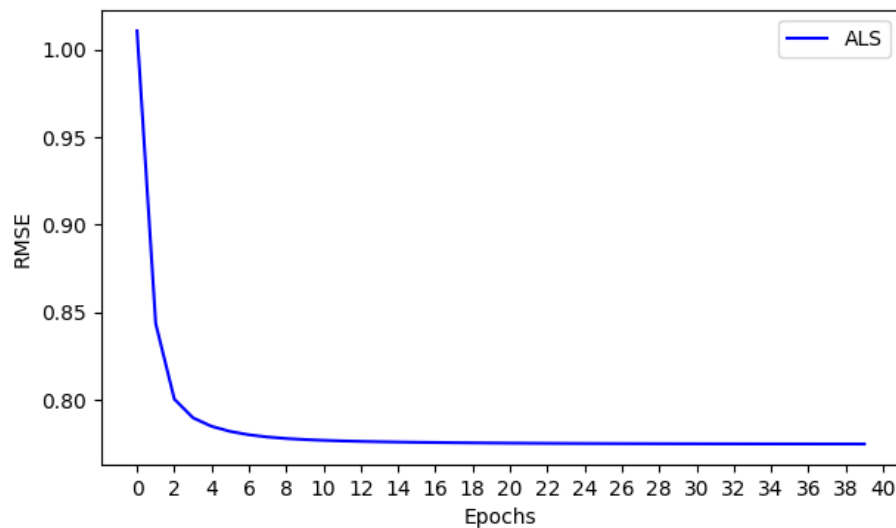


Рисунок 4.3 – Результаты обучения модели ALS

Из рисунка видно, что модель показала отличные результаты. Она достигла RMSE равное 0.78 за 5 итераций, а после обучение сильно замедлилось. Эти результаты показывают высокую точность и эффективность модели. Можно смело сказать, что реализация алгоритма ALS выполнена и модель показывает ожидаемо отличные результаты.

5 СРАВНЕНИЕ МОДЕЛЕЙ РЕКОМЕНДАТЕЛЬНЫХ СИСТЕМ

5.1 Датасет MovieLens

Все модели были протестированы на одном датасете - *MovieLens 1M Dataset*. *GroupLens Research* собрала и опубликовала наборы рейтинговых данных с веб-сайта *MovieLens*. Наборы данных были собраны за различные периоды времени, в зависимости от размера набора. Этот датасет содержит 1 миллион оценок от 6000 пользователей о 4000 фильмах.

```
[[4918 2808 1]
 [ 957 1660 4]
 [4653 914 5]
 [3245 3324 1]
 [5091 588 5]
 [ 704 969 3]
 [3940 3783 4]
 [2243 910 5]
 [2419 3274 2]]
```

Рисунок 5.1 – Пример оценок

На рисунке 5.1 показан пример 9 оценок. Датасет был преобразован в список из списков, где под нулевым индексом – индекс пользователя, под 1 индексом – индекс фильма и под 2 индексом – оценка фильма – натуральное число от 1 до 5 включительно.

5.2 Сравнение результатов работы моделей

На рисунке 5.1 видно, что после 30 итераций работы, лучше всего себя показала модель ALS с $RMSE=0.774$, на втором месте BPMF с $RMSE=0.805$ и на третьем месте PMF с $RMSE=0.909$. PMF после первой итерации показала $RMSE$ равный 1.1, а BPMF и ALS 1.1 и 1.0 соответственно.

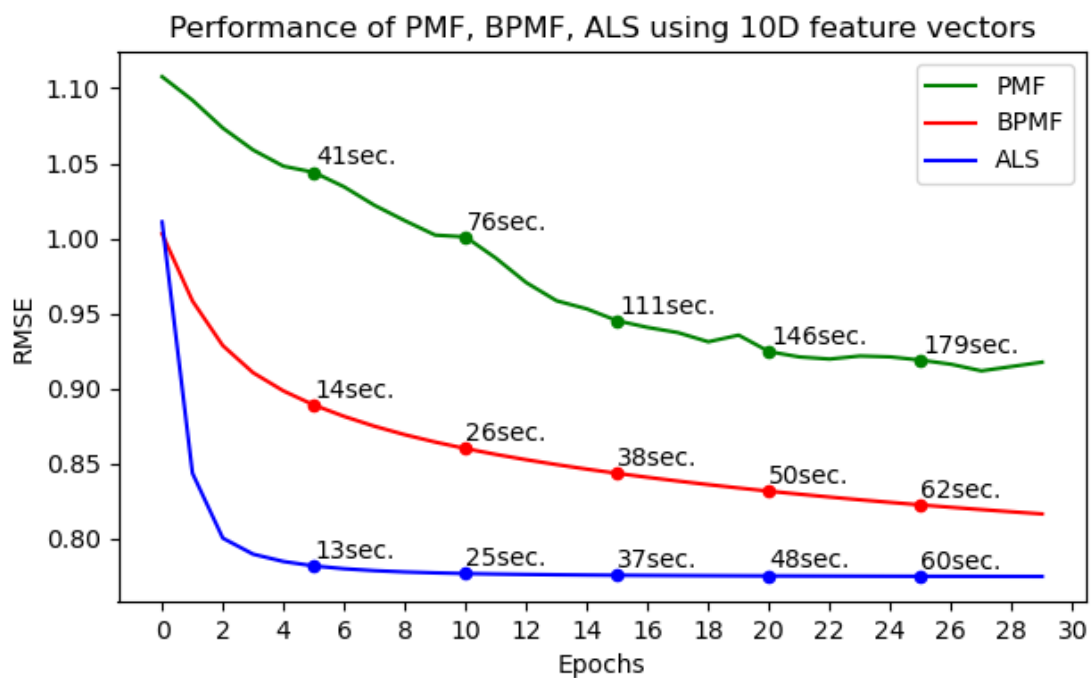


Рисунок 5.2 – Результаты тренировки трёх моделей
использующих 10-мерные вектора признаков

На рисунке 5.2 отчетливо видно, что после первых итераций модель ALS уже достигла RMSE лучше чем PMF и BPMF за всё время. PMF модель показала себя хуже всего. За 30 итераций она уменьшила RMSE на 0.19. График также показывает что обучение модели не проходило плавно. Модель BPMF после первой итерации показала хороший результат но обучение происходило медленнее чем у ALS. Модель ALS в первые пять итераций достигла сходимости, что говорит об отличном результате.

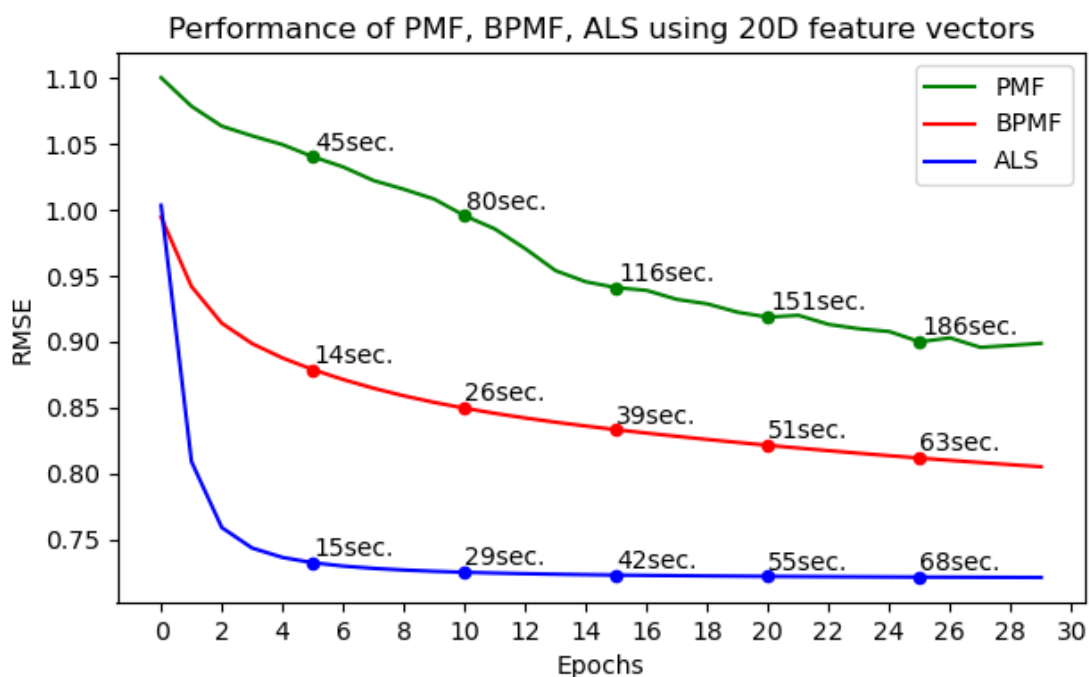


Рисунок 5.3 – Результаты тренировки трёх моделей
использующих 20-мерные вектора признаков

На рисунке 5.3 приведены результаты обучения моделей использующие 20-мерные вектора характеристик. По сравнению с прошлым примером, все модели улучшили RMSE. Также немного возросло время обучения моделей. Хотя разница может показаться не большой, на самом деле это серьёзное улучшение модели, которое обошлось нам в изменении одной настройки и немного времени.

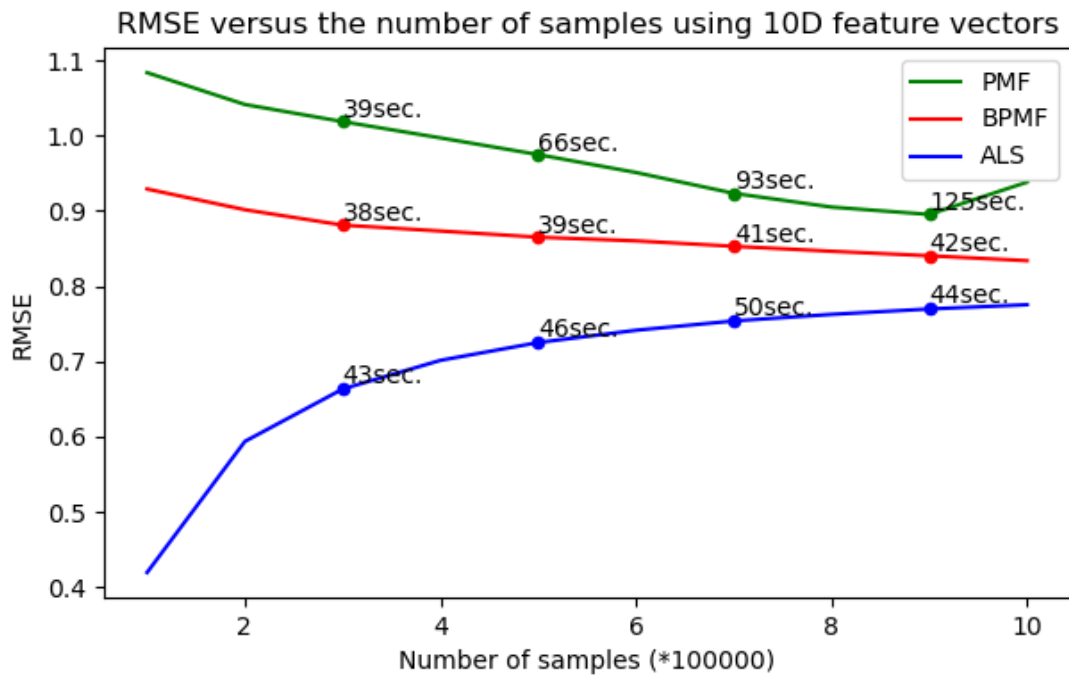


Рисунок 5.4 – График RMSE от количества оценок в датасете

На рисунке 5.4 приведены результаты обучения моделей использующих разное количество оценок. По оси x указано количество оценок от 100 000 до 1 000 000. По оси y указаны лучшее RMSE моделей обученных на таком количестве оценок. Из графика видно, что для PMF и BPMF увеличение количества оценок улучшает результат обучения. А для ALS наоборот. Но ALS в любом случае показывает лучшие результаты. Также отлично видно, что количество оценок не сильно повлияло на время работы BPMF и ALS, но время PMF увеличивалось линейно.

```
Input index of user: 1500
Input number of films to recomend: 5
Indexes of recommended films:
[535, 1135, 1942, 3138, 2124]
```

Рисунок 5.5 – Результаты тренировки трёх моделей

На рисунке 5.5 показан пример обращения к модели рекомендательной системы фильмов. Здесь задаются номер пользователя и количество фильмов нужных для рекомендации. А программа выводит для пользователя персональные рекомендации нужного количества фильмов.

ЗАКЛЮЧЕНИЕ

Настоящая выпускная квалификационная работа посвящена проблематике разработки рекомендательных систем с применением методов машинного обучения. В качестве предметной области выбраны системы рекомендации фильмов.

В работе рассмотрены 3 модели рекомендательных систем: с вероятностной матричной факторизации, байесовской вероятностной матричной факторизацией и крупномасштабной коллаборативной фильтрации.

Основные результаты полученные в работе:

- 1) предложены алгоритмы построения моделей рекомендательных систем: с вероятностной матричной факторизации, байесовской вероятностной матричной факторизацией и крупномасштабной коллаборативной фильтрации;
- 2) разработаны и протестированы программы реализации предложенных алгоритмов на языке python;
- 3) проведены вычислительные эксперименты с целью анализа работоспособности и эффективности предложенных алгоритмических решений;
- 4) проведен сравнительный анализ предложенных алгоритмов.

Лучший результат показал алгоритм ALS, этот факт можно объяснить большой разреженностью исходных данных. Работа может быть продолжена путем улучшения базовых алгоритмов, экспериментов с построением других гибридных моделей, использования дополнительных метаданных для создания систем, основанных на знаниях и систем на основе контента.

СПИСОК ИСТОЧНИКОВ

1. Ruslan Salakhutdinov, Andriy Mnih. Probabilistic Matrix Factorization // Neural Information Processing Systems – 2007. – P. 1 – 8.
2. Ruslan Salakhutdinov, Andriy Mnih. Bayesian Probabilistic Matrix Factorization using Markov Chain Monte Carlo // Proceedings of the 25th international conference on Machine learning – 2008. – P. 1 – 8.
3. Yunhong Zhou, Dennis Wilkinson, Robert Schreiber, Rong Pan. Large-scale Parallel Collaborative Filtering for the Netflix Prize // Algorithmic Aspects in Information and Management – 2008. – P. 1 – 12.
4. Radford M. Neal. Probabilistic inference using Markov chain Monte Carlo methods // Technical Report CRG-TR-93-1, Department of Computer Science, University of Toronto – 1993.
5. Raiko T., Ilin A., Karhunen, J.. Principal component analysis for large scale problems with lots of missing values // ECML – 2007. – P. 691–698.
6. R. Bell, Y. Koren, C. Volinsky. Modeling relationships at multiple scales to improve accuracy of large recommender systems // KDD – 2007. – P. 95–104.
7. M. Kurucz, A. A. Benczur, K. Csalogany. Methods for large scale svd with missing values // In Proc of KDD Cup and Workshop – 2007– P. 1–8.
8. K. Lang. NewsWeeder: Learning to filter netnews // In Proc. the 12th International Conference on Machine Learning, Tahoe City, CA, 1995– 2004.
9. S. Ghemawat, H. Gobioff, S.-T. Leung. The Google File System // In Proc. Of SOSP'03: 19th ACM Symposium on Operating Systems Principles – 2003. – P. 29–43.
10. Aggarwal C. C. Data mining. The Textbook. Springer International Publishing, 2015. 734 p.
11. Mango T., Sable C. A comparison of signal-based music recommendation to genre labels, collaborative filtering, musicological analysis, human recommendation, 42 and random baseline // Proceedings of the 9th international conference on music information retrieval, 2008. P. 161-166.

12. Knees P., Pohle T., Schedl M., Widmer G. Combining audio-based similarity with web-based data to accelerate automatic music playlist generation // Proceedings of the 8th ACM international workshop on Multimedia information retrieval, 2006. P. 147-154.
13. Schedl M., Flexer A., Urbano J. The neglected user in music information retrieval research // Journal of Intelligent Information Systems, 2013. Vol. 41, Issue 3. P. 523-539.
14. Gabriel H. H., Spiliopoulou M., Nanopoulos A. Eigenvectop-based clustering using aggregated similarity matrices // Proceedings of the 2010 ACM Symposium on Applied Computing, 2010. P. 1083-1087.
15. Королева Д. Е., Филиппов М. В. Анализ алгоритмов обучения коллаборативных рекомендательных систем // Инженерный журнал: наука и инновации, 2013. Вып. 6. Стр. 1-8.
16. Николенко С.А. Рекомендательные системы. СПб: Изд-во Центр Речевых Технологий, 2012. 53 с.
17. Berry M.W. Large scale singular value computations // International Journal of Supercomputer Applications, 1992. No. 6(1). P. 13–49.
18. Billsus D., Pazzani M.J. Learning Collaborative Information Filters // Proceeding 15th International Conference on Machine Learning, 1998. P. 46-54.
19. Ekstrand M. D., Riedl J. T., Konstan J. A. Collaborative Filtering Recommender Systems // Foundations and Trends® in Human–Computer Interaction, 2011. Vol. 4, No. 2. P. 81-173.
20. Goldberg K., Roeder T., Gupta D., Perkins C. Eigentaste: A constant time collaborative filtering algorithm // Information Retrieval, 2001. Vol. 4, No. 2. P. 133–151. 43
21. Aggarwal C. C. Recommender Systems. The Textbook. Springer International Publishing, 2016. 498 p.
22. Джонс М. Рекомендательные системы.
<https://www.ibm.com/developerworks/ru/library/os-recommender1/index.html>

23. Pazzani M., Billsus D. Learning and revising user profiles: The identification of interesting web sites // Machine Learning - Special issue on multistrategy learning, 1997. Vol. 27, Issue 3. P. 313–331.
24. Jannach D., Zanker M., Felfernig A., Friedrich G. Recommender Systems – An Introduction. Cambridge University Press, 2010. 360. P
25. Herlocker J., Webster J., Jung S., Dragunov A., Holt T., Culter T., Haerer S. A framework for collaborative information environments and unified access to distributed digital content // Proceedings of the 2nd ACM/IEEE-CS joint conference on Digital libraries, 2002. P. 378-378.
26. Wang A. The Shazam music recognition service // Communications of the ACM, 2006. P. 44-48.
27. Vucetic S., Obradovic Z. Performance Controlled Data Reduction for Knowledge Discovery in Distributed Databases // Proceedings of the 4th Pacific-Asia Conference on Knowledge Discovery and Data Mining, Current Issues and New Applications, 2000. P. 29-39.
28. Sarwar B., Karypis G., Konstan J., Riedl J. Analysis of recommendation algorithms for e-commerce // Proceedings of the 2nd ACM conference on Electronic commerce, 2000. P. 158-167.
29. Zaharia M., Chowdhury M., Franklin M. J., Shenker S., Stoica I. Spark: cluster computing with working sets // HotCloud, 2010. Vol. 10, P. 10.

ПРИЛОЖЕНИЕ А

Ссылка на код

